

Build Your Own Search Engine

Week 1 · Day 3 — RAII & Smart Pointers

C++ Memory Safety for Production Crawler Code

Day 3 Objective: Replace raw `new/delete` from Days 1–2 with RAII-based smart pointers. Every file handle, socket, and buffer in your crawler will use these patterns—learning them now prevents memory leaks in a long-running 64-thread crawler.

Contents

1	RAII — Resource Acquisition Is Initialisation	2
1.1	The Core Principle	2
1.2	RAII Pattern: File Handle	2
2	Smart Pointers	3
2.1	<code>unique_ptr</code> — Exclusive Ownership	3
2.2	<code>shared_ptr</code> — Shared Ownership (Reference Counting)	3
2.3	<code>weak_ptr</code> — Non-Ownning Observer	4
2.4	Choosing the Right Pointer	4
3	Move Semantics and <code>std::move</code>	4
3.1	The Problem Move Solves	5
3.2	The Rule of Five (Modern C++)	5
4	RAII for Mutex Locks	5
5	Practical Patterns in the Crawler	6
5.1	Socket RAII Wrapper	6
5.2	Thread-Pool Task with <code>unique_ptr</code>	6
6	Common Mistakes and Fixes	7
7	Build Tasks	7
8	Quick Reference Cheat Sheet	8
9	Day 3 Output Checklist	8

1 RAII — Resource Acquisition Is Initialisation

1.1 The Core Principle

RAII ties the lifetime of a resource (memory, file handle, mutex lock, socket) directly to the lifetime of a C++ object:

- **Constructor** acquires the resource.
- **Destructor** releases it — automatically, even on exceptions.
- The resource *cannot* outlive the object; the object *cannot* exist without the resource.

Why RAII matters for your crawler

Your crawler runs 64 threads 24/7, each opening sockets, reading files, and locking mutexes. A single forgotten `delete` or unclosed socket causes a slow memory/FD leak that only manifests after hours of running. RAII makes leaks *structurally impossible*—the destructor always runs.

1.2 RAII Pattern: File Handle

Listing 1: RAII file wrapper — used for saving crawled HTML to disk

```

1 // Without RAII -- leaks on any early return or exception
2 FILE* f = fopen("page.html", "w");
3 write_html(f);           // throws? -> f never closed!
4 fclose(f);               // may never execute
5
6 // With RAII
7 class FileHandle {
8     FILE* fp_;
9 public:
10     explicit FileHandle(const char* path, const char* mode)
11         : fp_(fopen(path, mode)) {
12         if (!fp_) throw std::runtime_error("Cannot open file");
13     }
14     ~FileHandle() { if (fp_) fclose(fp_); }    // ALWAYS runs
15
16     // Delete copy; allow move
17     FileHandle(const FileHandle&) = delete;
18     FileHandle& operator=(const FileHandle&) = delete;
19
20     FILE* get() const { return fp_; }
21 };
22
23 // Usage -- no manual close needed, exception-safe
24 {
25     FileHandle fh("page.html", "w");
26     fputs(html_content.c_str(), fh.get());
27 } // <-- destructor runs here, file closed

```

Search Engine Connection

Your crawler saves every fetched HTML blob as a content-addressed file (SHA256(body) as filename, similar to MinIO). Wrapping `fopen` in an RAII handle means every write path is leak-proof even when the network throws an exception mid-write.

2 Smart Pointers

Smart pointers are the standard RAII wrappers for heap memory. All three live in `<memory>`.

2.1 `unique_ptr` — Exclusive Ownership

`unique_ptr` in one sentence

Exactly *one* owner. When the owner goes out of scope, the memory is freed. Zero runtime overhead vs. a raw pointer—the compiler optimises it away entirely.

Listing 2: `unique_ptr` basics and move semantics

```
1 #include <memory>
2
3 // Creation -- prefer make_unique (C++14), never raw new
4 auto buf = std::make_unique<char[]>(4096);    // heap buffer
5
6 // Ownership transfer with std::move
7 std::unique_ptr<char[]> take_ownership(std::unique_ptr<char[]> p) {
8     // do work with p
9     return p;    // move out -- original pointer becomes null
10 }
11
12 auto owned = take_ownership(std::move(buf));
13 // buf is now null; owned has the memory
14
15 // Custom deleter -- for non-memory resources
16 auto sock = std::unique_ptr<int, decltype(&close)>(
17     new int(socket(AF_INET, SOCK_STREAM, 0)), close);
18 // socket closed automatically when sock goes out of scope
```

Key rules:

- `unique_ptr` is *move-only*—you cannot copy it.
- Use `std::move()` to transfer ownership.
- After `std::move(p)`, `p` is `nullptr`—check before use.
- Use custom deleters for sockets, file descriptors, and other OS handles.

2.2 `shared_ptr` — Shared Ownership (Reference Counting)

`shared_ptr` in one sentence

Multiple owners share the same heap object. An atomic reference count tracks owners; memory freed when count reaches 0.

Listing 3: `shared_ptr` — shared DNS cache entry

```
1 struct DNSEntry {
2     std::string ip;
3     std::chrono::steady_clock::time_point expiry;
4 };
5
6 // Multiple threads can hold a shared_ptr to the same cache entry
7 auto entry = std::make_shared<DNSEntry>("142.250.80.46",
8     std::chrono::steady_clock::now() +
9     std::chrono::seconds(300));
10 std::shared_ptr<DNSEntry> copy1 = entry;    // ref count: 2
```

```

11 std::shared_ptr<DNSEntry> copy2 = entry;    // ref count: 3
12
13 // Inspect ref count (for debugging only)
14 std::cout << entry.use_count() << "\n";    // prints 3
15
16 copy1.reset();    // ref count: 2
17 copy2.reset();    // ref count: 1
18 // entry still alive; freed only when entry itself goes out of scope

```

Overhead Warning

`shared_ptr` has **two heap allocations** (object + control block) unless you use `make_shared`, plus an **atomic increment/decrement** on every copy. In your thread-pool with 64 threads copying shared pointers in tight loops, this overhead adds up. **Prefer `unique_ptr`; reach for `shared_ptr` only when multiple owners truly exist.**

2.3 weak_ptr — Non-Owning Observer

`weak_ptr` observes a `shared_ptr`-managed object *without* incrementing the reference count. Its main purpose: **break reference cycles**.

Listing 4: Breaking a cycle with `weak_ptr`

```

1 // Reference cycle -- memory leak!
2 struct Node {
3     std::shared_ptr<Node> next;    // strong ref
4 };
5
6 // Fix: make one direction weak
7 struct SafeNode {
8     std::shared_ptr<SafeNode> next;    // strong -- owns forward
9     std::weak_ptr<SafeNode> prev;    // weak -- does NOT own backward
10 };
11
12 // Usage
13 auto a = std::make_shared<SafeNode>();
14 auto b = std::make_shared<SafeNode>();
15 a->next = b;    // a owns b (strong)
16 b->prev = a;    // b observes a (weak -- no cycle)
17
18 // To use a weak_ptr, lock() it first
19 if (auto locked = b->prev.lock()) {    // returns shared_ptr or null
20     std::cout << "prev still alive\n";
21 }

```

2.4 Choosing the Right Pointer

Pointer	Owners	Overhead	Use Case in Crawler
<code>unique_ptr</code>	1	Zero	Thread-local HTTP fetcher, socket
<code>shared_ptr</code>	≥ 1	Atomic refcount	DNS cache entries shared across threads
<code>weak_ptr</code>	0	Minimal	Back-references in link graph
Raw pointer	0	Zero	Non-owning observer (use carefully)

3 Move Semantics and `std::move`

3.1 The Problem Move Solves

Copying is expensive for large objects (HTML buffers, posting lists). Move semantics let you *steal* the contents of a temporary object instead of copying them.

Listing 5: Move vs. copy — critical for zero-copy HTML passing

```

1  std::string html = fetch_page(url);    // 200 KB string
2
3  // COPY -- allocates 200 KB, copies every byte
4  std::string copy_html = html;         // expensive!
5
6  // MOVE -- transfers the internal pointer, O(1)
7  std::string moved_html = std::move(html);
8  // html is now empty (valid but unspecified state)
9  // moved_html owns the 200 KB buffer
10
11 // In your crawler: pass HTML to Kafka producer without copying
12 void send_to_kafka(std::string html) { /* ... */ }
13 send_to_kafka(std::move(raw_html));   // zero copy

```

3.2 The Rule of Five (Modern C++)

If you define *any* of these, define *all five* (or = delete/= default them explicitly):

Special Member	Signature	When Triggered
Destructor	~T()	Object goes out of scope
Copy constructor	T(const T&)	T b = a;
Copy assignment	T& operator=(const T&)	b = a;
Move constructor	T(T&&)	T b = std::move(a);
Move assignment	T& operator=(T&&)	b = std::move(a);

4 RAII for Mutex Locks

Your URL queue and DNS cache are protected by mutexes. Never call `mutex.lock()` directly—use RAII wrappers.

Listing 6: RAII mutex wrappers — safe queue access in thread pool

```

1  #include <mutex>
2
3  std::mutex url_queue_mutex;
4  std::queue<std::string> url_queue;
5
6  // BAD -- unlock never called on exception
7  url_queue_mutex.lock();
8  auto url = url_queue.front();
9  url_queue.pop();
10 url_queue_mutex.unlock();    // skipped if exception above!
11
12 // GOOD -- lock_guard (scoped, non-transferable)
13 {
14     std::lock_guard<std::mutex> lock(url_queue_mutex);
15     auto url = url_queue.front();
16     url_queue.pop();
17 }    // <-- lock released here, guaranteed
18
19 // BETTER for DNS cache -- shared_mutex (readers/writers)
20 #include <shared_mutex>
21 std::shared_mutex dns_mutex;

```

```

22
23 // Many reader threads -- concurrent reads allowed
24 std::shared_lock<std::shared_mutex> read_lock(dns_mutex);
25
26 // Single writer thread -- exclusive access
27 std::unique_lock<std::shared_mutex> write_lock(dns_mutex);

```

Search Engine Connection

Your DNSResolver from Day 7 caches IP addresses with a TTL. Multiple crawler threads read it concurrently (`shared_lock`), but only one writes on a cache miss (`unique_lock`). This exact RAII pattern is used in the blueprint's DNSResolver class.

5 Practical Patterns in the Crawler

5.1 Socket RAII Wrapper

Listing 7: RAII socket — blueprint HTTPFetcher::fetch() uses this pattern

```

1 class Socket {
2     int fd_;
3 public:
4     Socket() : fd_(::socket(AF_INET, SOCK_STREAM, 0)) {
5         if (fd_ < 0) throw std::runtime_error("socket() failed");
6     }
7     ~Socket() { if (fd_ >= 0) ::close(fd_); }
8
9     // Move-only
10    Socket(Socket&& o) noexcept : fd_(o.fd_) { o.fd_ = -1; }
11    Socket& operator=(Socket&& o) noexcept {
12        if (this != &o) { ::close(fd_); fd_ = o.fd_; o.fd_ = -1; }
13        return *this;
14    }
15    Socket(const Socket&) = delete;
16    Socket& operator=(const Socket&) = delete;
17
18    int get() const { return fd_; }
19    int release() { int tmp = fd_; fd_ = -1; return tmp; }
20 };
21
22 // Usage -- socket closed automatically, even on exception
23 void fetch(const std::string& host, int port) {
24     Socket sock; // acquired
25     connect_to(sock.get(), host, port); // may throw
26     send_http_get(sock.get(), host);
27     recv_response(sock.get());
28 } // <-- sock.~Socket() called here

```

5.2 Thread-Pool Task with unique_ptr

Listing 8: Thread pool enqueue using `unique_ptr` for zero-copy task passing

```

1 // Task wrapper -- moves the fetch task into the queue
2 struct FetchTask {
3     std::string url;
4     std::unique_ptr<HTTPFetcher> fetcher;
5 };
6
7 std::queue<std::unique_ptr<FetchTask>> task_queue;
8
9 // Enqueue -- no copy of FetchTask or its fetcher

```

```

10 auto task = std::make_unique<FetchTask>();
11 task->url    = "https://example.com/page.html";
12 task->fetcher = std::make_unique<HTTPFetcher>();
13 task_queue.push(std::move(task));    // O(1), no copy
14
15 // Worker thread dequeues and processes
16 auto t = std::move(task_queue.front());
17 task_queue.pop();
18 t->fetcher->fetch(t->url);
19 // t goes out of scope -- FetchTask and its fetcher freed automatically

```

6 Common Mistakes and Fixes

Mistake	Fix
Using <code>p.get()</code> and storing the raw pointer elsewhere	Never store a raw pointer longer than the smart pointer lives.
Passing <code>unique_ptr</code> by value to a function	Pass by reference if no ownership transfer; <code>std::move()</code> to transfer.
Creating <code>shared_ptr</code> from the same raw pointer twice	Always use <code>make_shared<>()</code> —never <code>shared_ptr<T>(raw_ptr)</code> twice.
Returning a local <code>unique_ptr</code> with <code>std::move()</code>	Unnecessary—NRVO handles it; just <code>return ptr</code> ;
Cycling <code>shared_ptrs</code> in graph structures	Use <code>weak_ptr</code> for back-edges (link graph, prev pointers).

7 Build Tasks

Build Tasks — Day 3

- Rewrite Day 1 array using `unique_ptr`**
Replace your `new int[N]/delete[]` dynamic array with `std::unique_ptr<int[]>`. Verify with Valgrind: `valgrind -leak-check=full ./array_test → zero leaks`.
- `shared_ptr` reference count graph**
Create a `shared_ptr<string>`, make 3 copies, print `use_count()` after each copy and each `reset()`. Draw the count timeline.
- Dangling pointer bug → fix with smart pointer**
Write a function that returns a raw pointer to a local variable. Demonstrate the crash/UB, then fix by returning `unique_ptr`.
- RAII file-handle wrapper**
Implement the `FileHandle` class from Section 1.2. Test: constructor failure throws; destructor always closes; move transfers ownership. Wire it into your Day 6 SHA256 content-addressed HTML writer.
- Socket RAII wrapper (project task)**
Implement the `Socket` class from Section 5.1. Verify with `lsof | grep your_binary` that no FDs leak across 100 sequential connects.
- GoogleTest unit tests (Week 11 preparation)**
Write tests for `FileHandle`: open succeeds, open fails throws, move transfers FD, destructor closes FD. Run with `cmake -build . -target test`.

8 Quick Reference Cheat Sheet

Operation	Syntax	Notes
Create <code>unique_ptr</code>	<code>make_unique<T>(args...)</code>	Preferred over <code>new</code>
Create <code>shared_ptr</code>	<code>make_shared<T>(args...)</code>	Single allocation
Transfer ownership	<code>std::move(p)</code>	Leaves <code>p</code> null
Non-owning borrow	<code>p.get()</code>	Raw pointer, short-lived
Release ownership	<code>p.release()</code>	Returns raw, <code>p</code> null
Swap two pointers	<code>p.swap(q)</code>	Both remain valid
Reset / free	<code>p.reset()</code>	Frees immediately
Observe <code>shared_ptr</code>	<code>weak_ptr<T> w = sp</code>	No ownership
Lock a <code>weak_ptr</code>	<code>auto sp = w.lock()</code>	Returns null if expired
Scoped lock	<code>lock_guard<mutex> lg(m)</code>	RAII, non-movable
Unique lock	<code>unique_lock<mutex> ul(m)</code>	RAII, movable, <code>condvar</code>
Shared (read) lock	<code>shared_lock<shared_mutex></code>	Multiple concurrent readers

9 Day 3 Output Checklist

At the end of Day 3, your repo should contain:

- ✓ `unique_ptr`-based dynamic array (Day 1 rewrite)
- ✓ `FileHandle` RAII class with unit tests (GoogleTest)
- ✓ `Socket` RAII class (feeds directly into Week 3 HTTP client)
- ✓ Zero Valgrind leaks on all Day 3 programs
- ✓ Reference-count timeline diagram committed to repo docs

These files **directly reuse** in: Week 3 (`Socket`), Week 4 (`FileHandle` for HTML storage), Week 7 (move semantics for zero-copy posting list passing), and Week 11 (the GoogleTest fixtures you write today become templates for all future tests).